

ヒープの構築と再構築（ヒープ化）

ツリー構造のところでお話ししたヒープの特徴を復習しましょう。

「**ヒープ (Heap)**」は二分木をベースにしたデータ構造です。

「ヒープ（積み上げた山）」という名前の通り、「**一番大きい要素を常にすぐ取り出せるようにしておく**」という目的に特化しています。

一番小さい要素をすぐに取り出せるようにした「最小ヒープ」もありますが、ここでは扱いません。

ヒープとして成立するためには、以下の2つのルールを同時に満たす必要があります。

1. 親子関係のルール：親ノードの値は、子ノードの値よりも必ず大きい（または等しい）。左右の子どものどちらが大きいかにはルールはありません。
2. 形のルール（完全二分木）：上から順番に隙間なくノードが詰まっていて、最下位の段は必ず左詰めでデータが入られます。



ヒープの構築

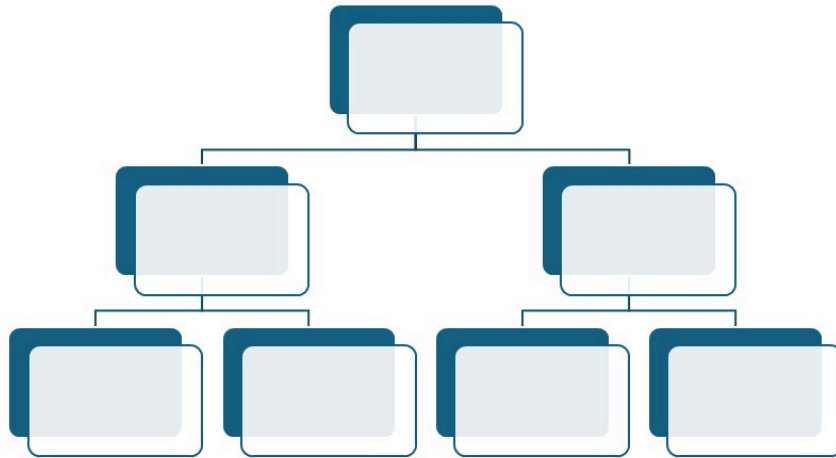
では、ここからヒープを構築する手順について、説明します。

例として、[5, 2, 7, 3, 6, 1, 4] というバラバラに並んだ配列をツリーに追加して、「親が子より大きい」というルールにしたがって、ヒープを構築します。

Array

0	1	2	3	4	5	6
5	2	7	3	6	1	4

Heap



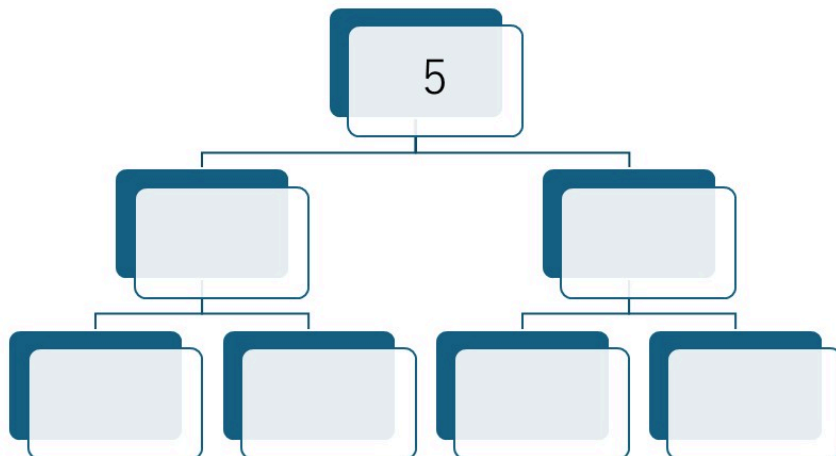
- 5 をツリーに追加

Array

0	1	2	3	4	5	6
	2	7	3	6	1	4

Heap

降順になるように
ヒープを構築する



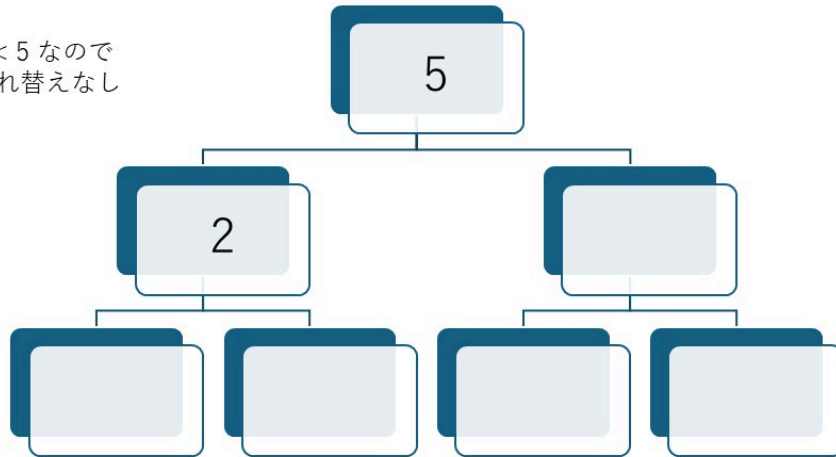
- 2 を追加 → $2 < 5$ なので、安定

Array

0	1	2	3	4	5	6
		7	3	6	1	4

Heap

$2 < 5$ なので
入れ替えなし



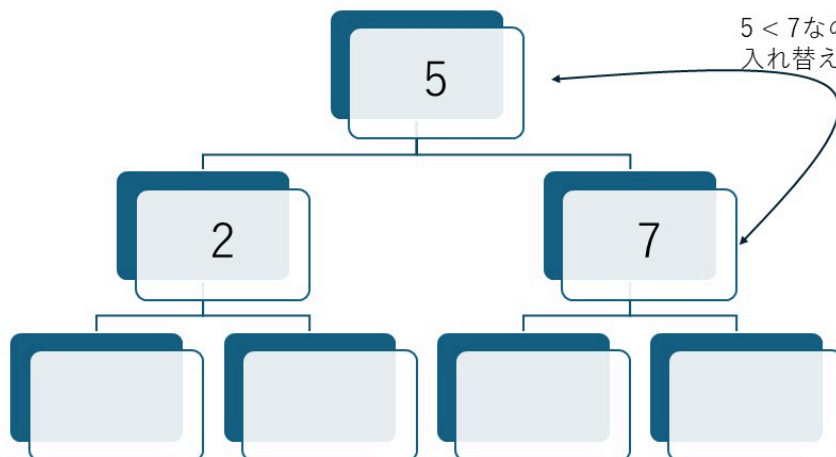
- 7 を追加 → $5 < 7$ なので、入れ替え

Array

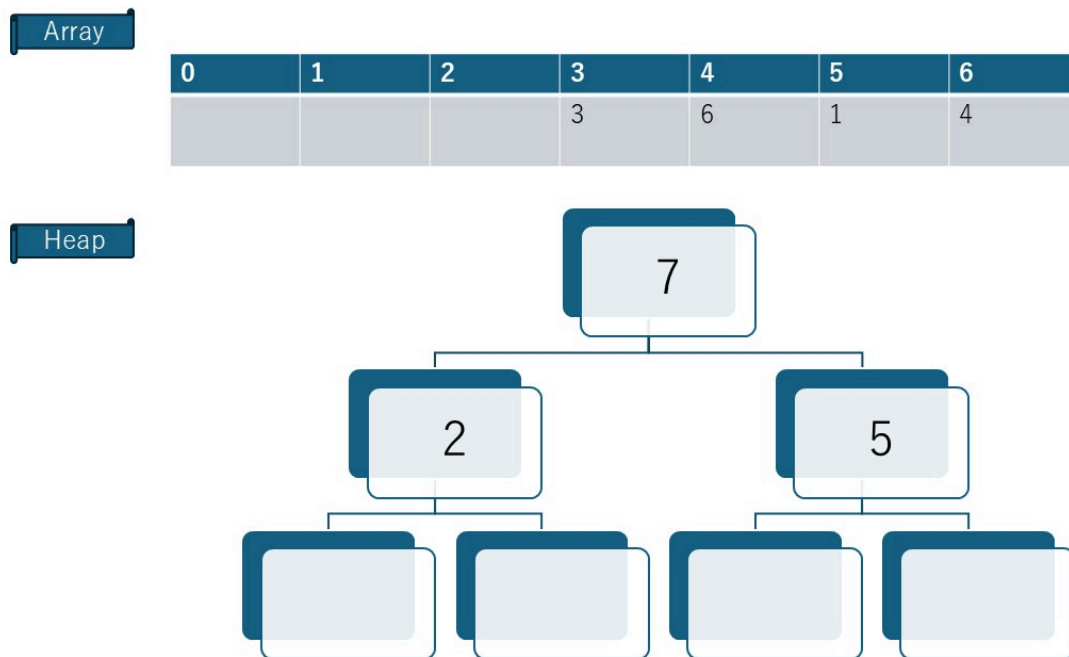
0	1	2	3	4	5	6
			3	6	1	4

Heap

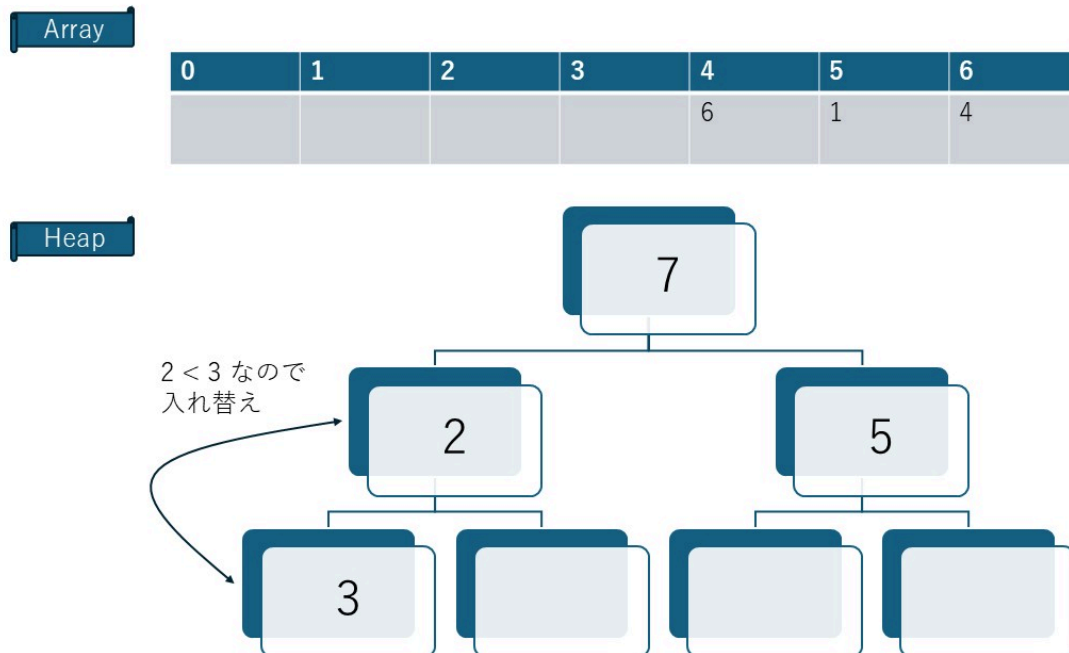
$5 < 7$ なので
入れ替え



- 入れ替え後、親 > 子のルールに従っているので、**安定**



- 3 を追加 → $2 < 3$ なので、**入れ替え**

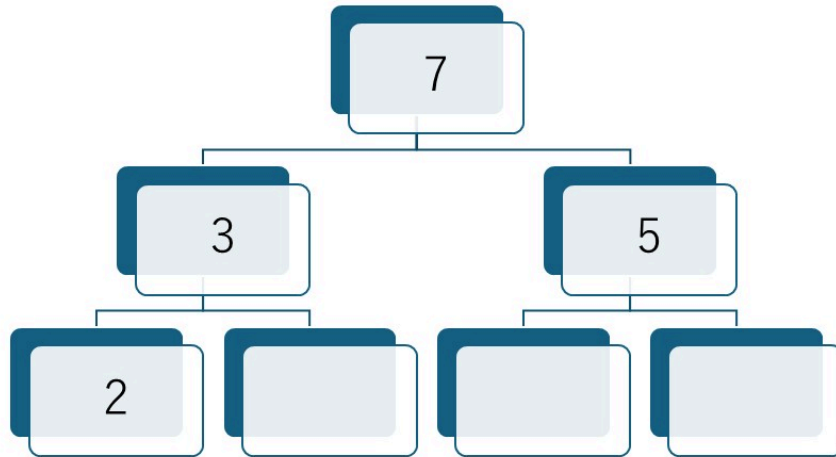


- 入れ替え後、親 > 子のルールに従っているので、**安定**

Array

0	1	2	3	4	5	6
				6	1	7

Heap

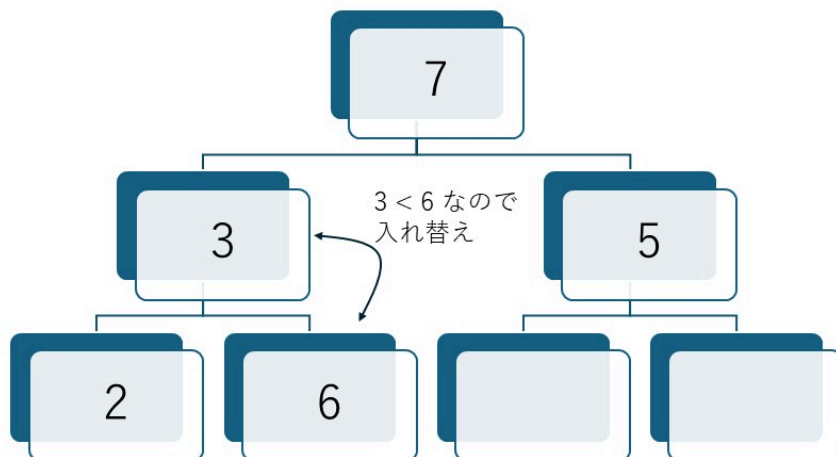


- 6 を追加 → $3 < 6$ なので、**入れ替え**

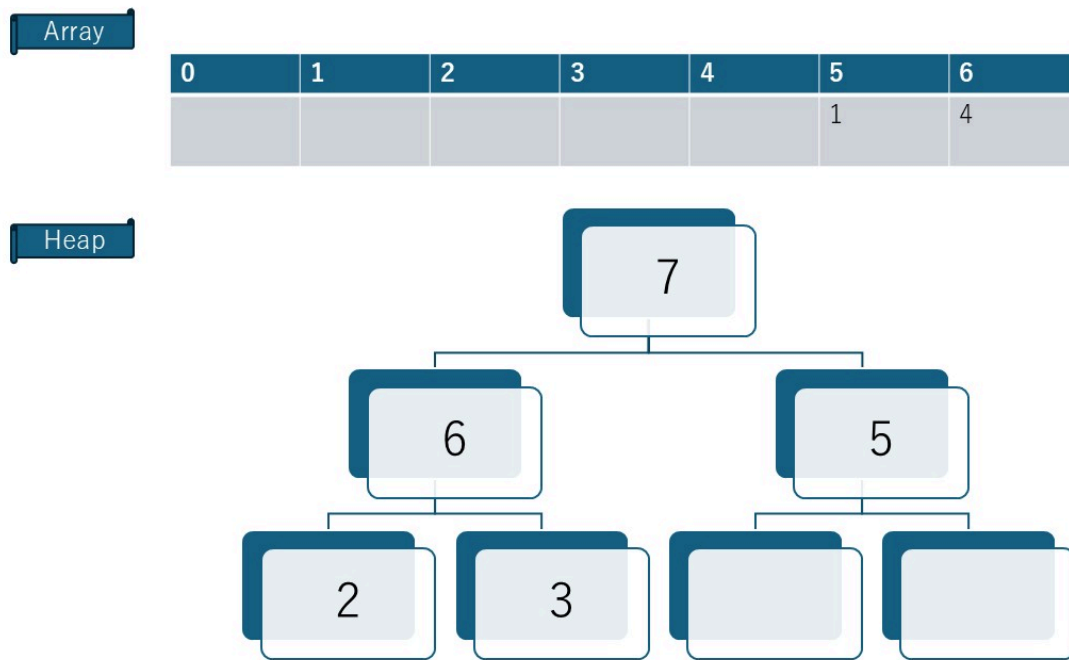
Array

0	1	2	3	4	5	6
					1	4

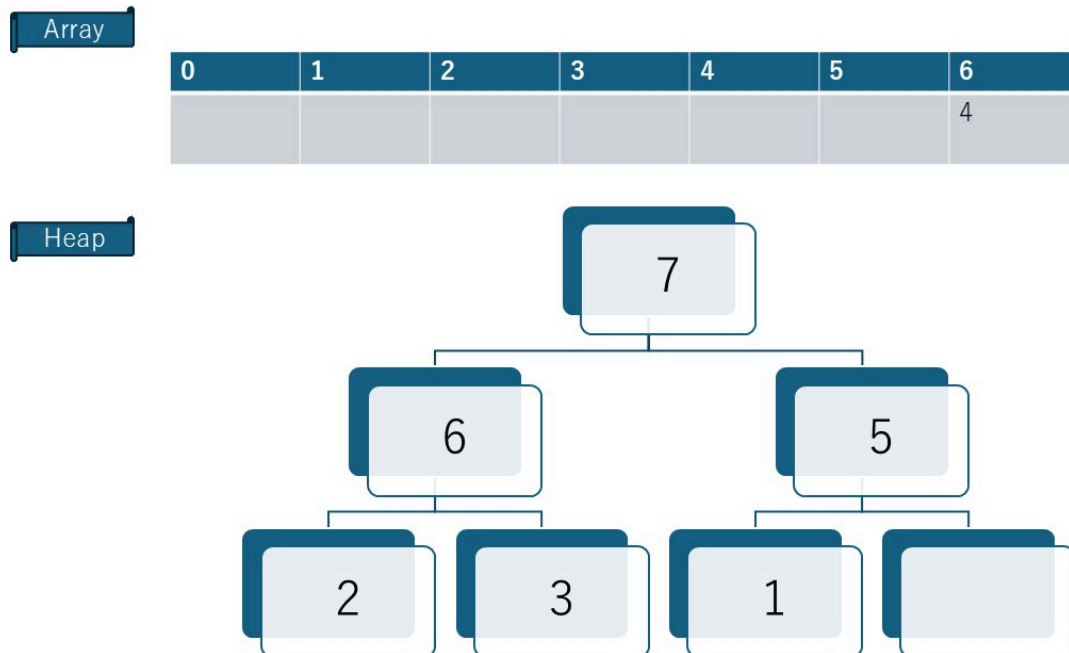
Heap



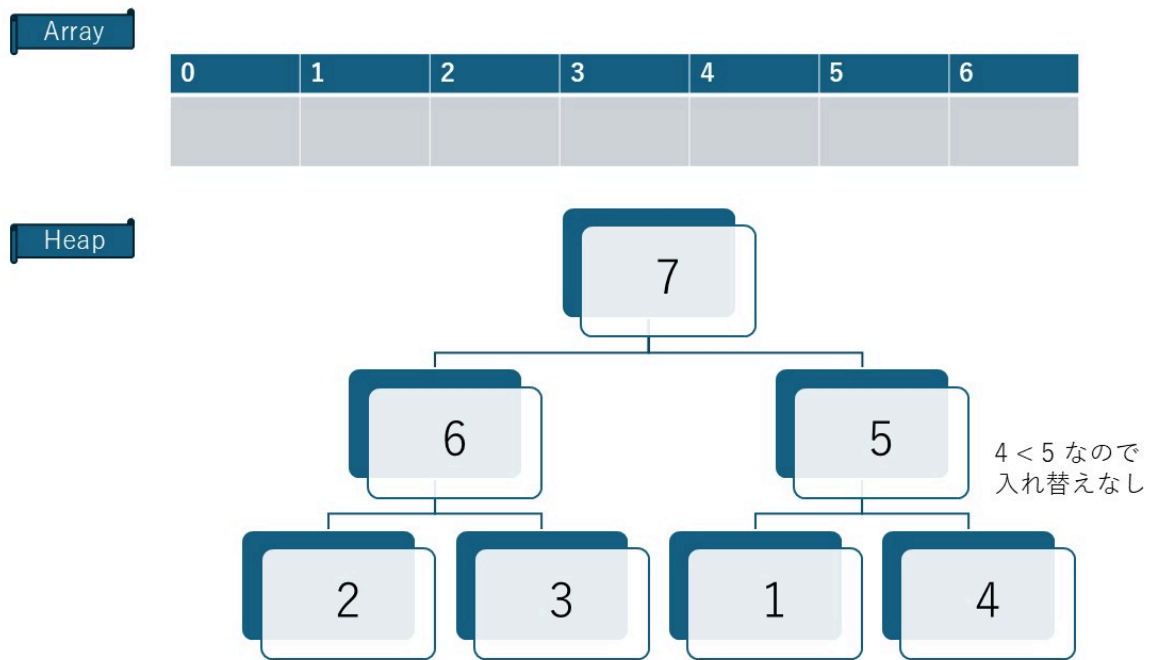
- 入れ替え後、親 > 子のルールに従っているので、**安定**



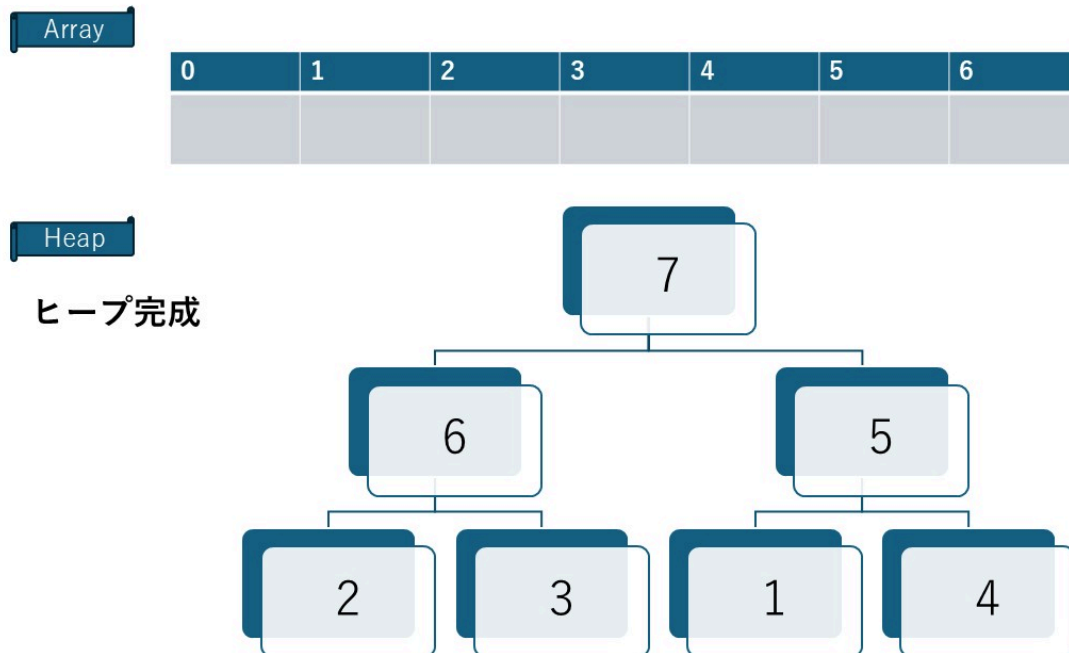
- 1 を追加 → $1 < 5$ なので、**安定**



- 4 を追加 → $4 < 5$ なので、安定



- これで、ヒープ完成!



ヒープの再構築：ヒープ化 (Heapify)

データを削除（頂点を取り出す）したとき、ヒープは以下の手順で形を整えます。

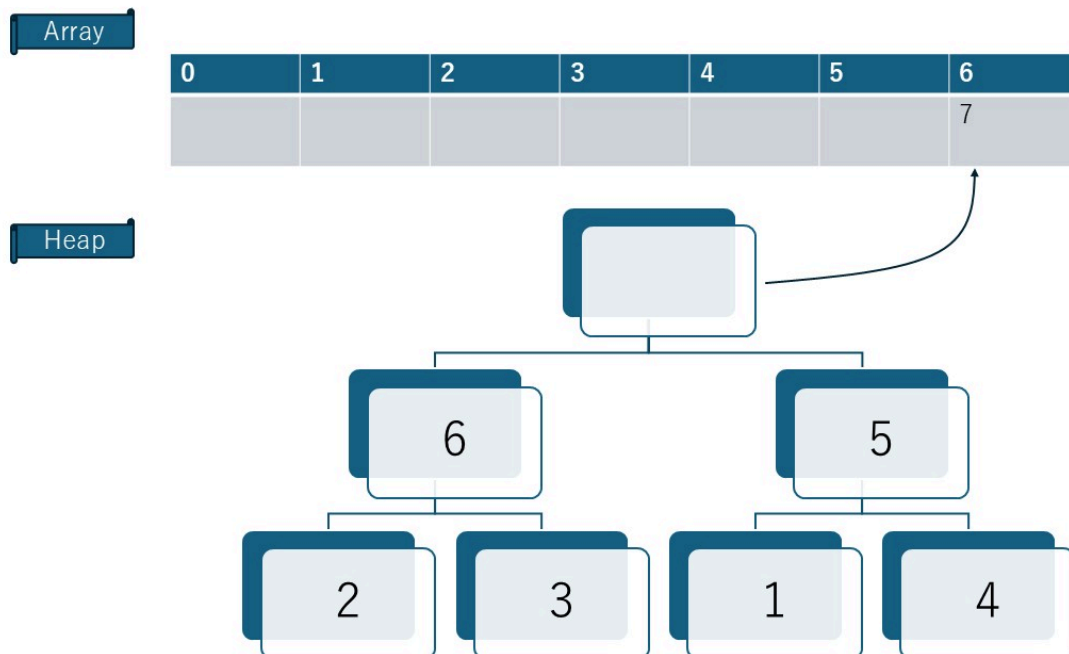
1. 空いた頂点に、一番末尾（右下）の要素を持ってくる。
2. その要素を子と比較し、ルール違反（子が自分より大きい等）なら入れ替える。
3. これを適切な位置に収まるまで繰り返す（沈み込み）。

これを使って、配列をソートすることができます（ヒープソート）。

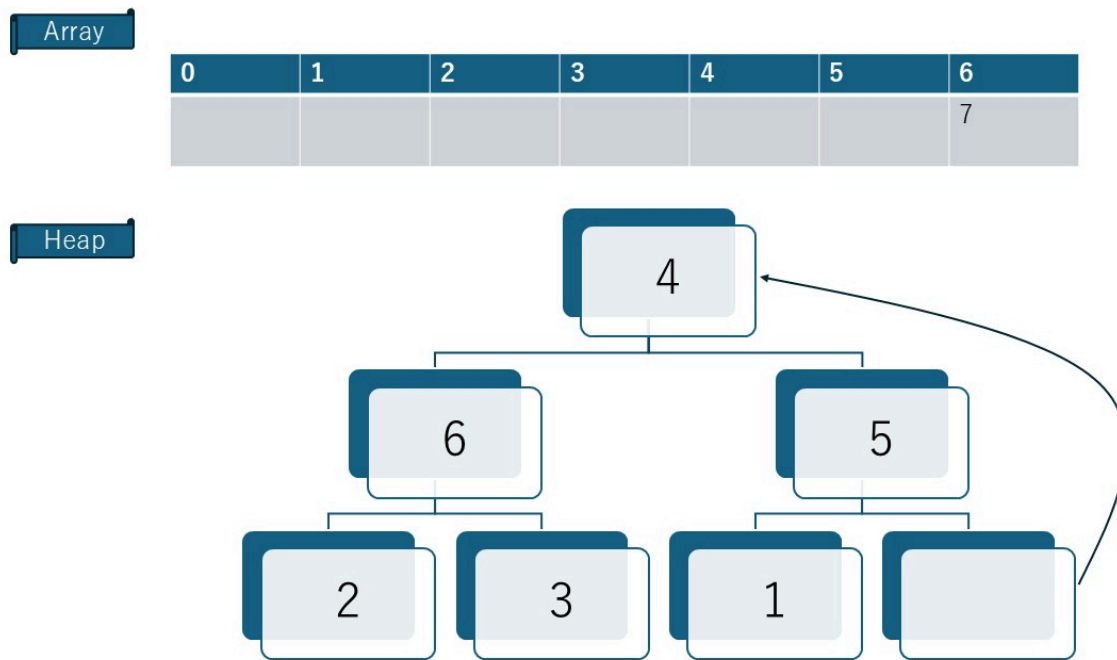
![note]

ヒープソートの実装については、「ヒープソート」の章でくわしく説明します。ここでは、「ヒープの再構築」の例として図解します。

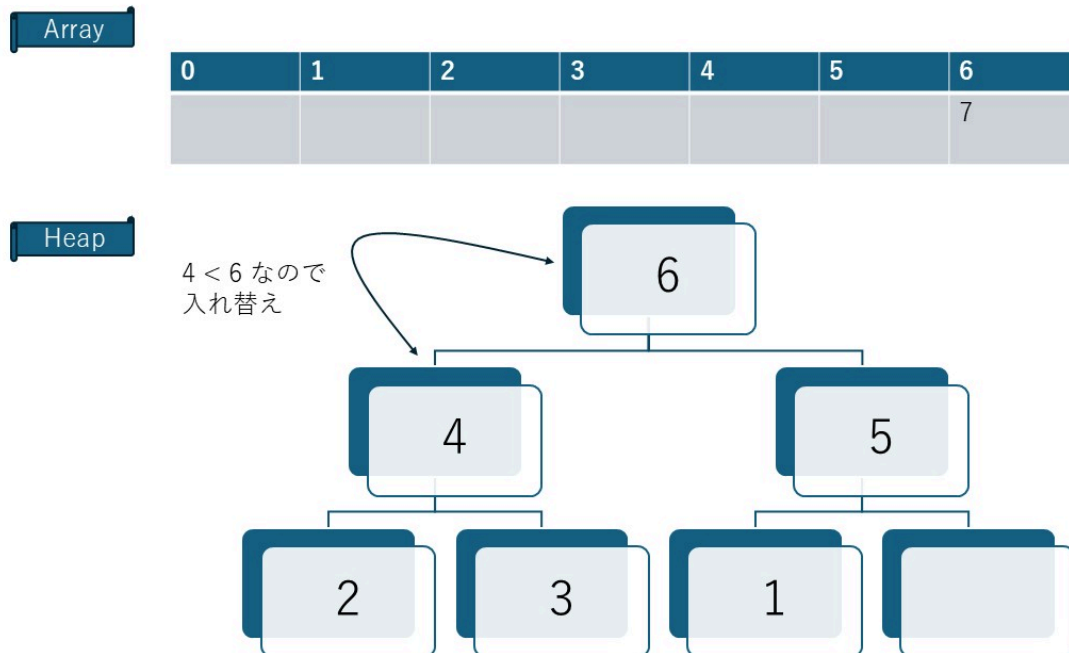
- 頂点の 7 を取り出す



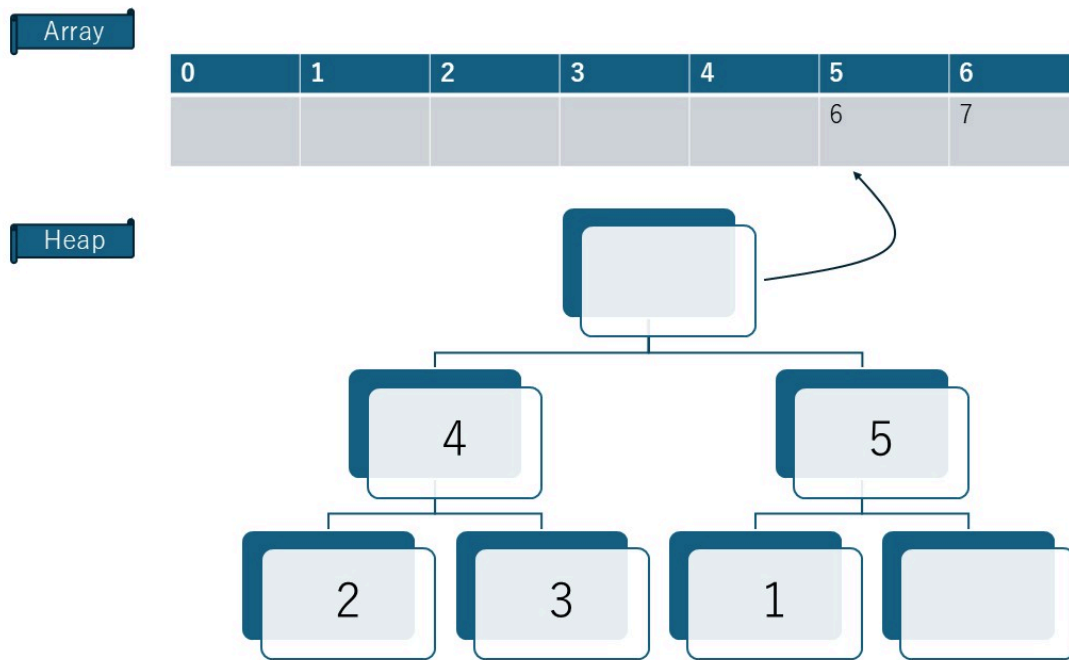
- 右下の 4 を空いた頂点に移動



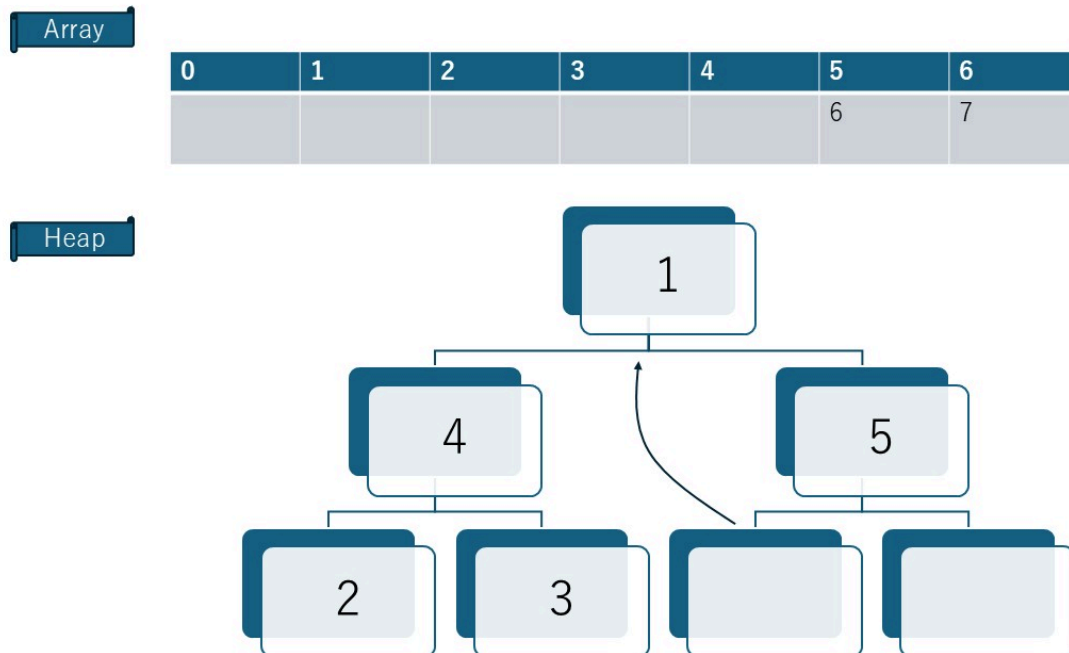
- $4 < 6$ なので、入れ替え



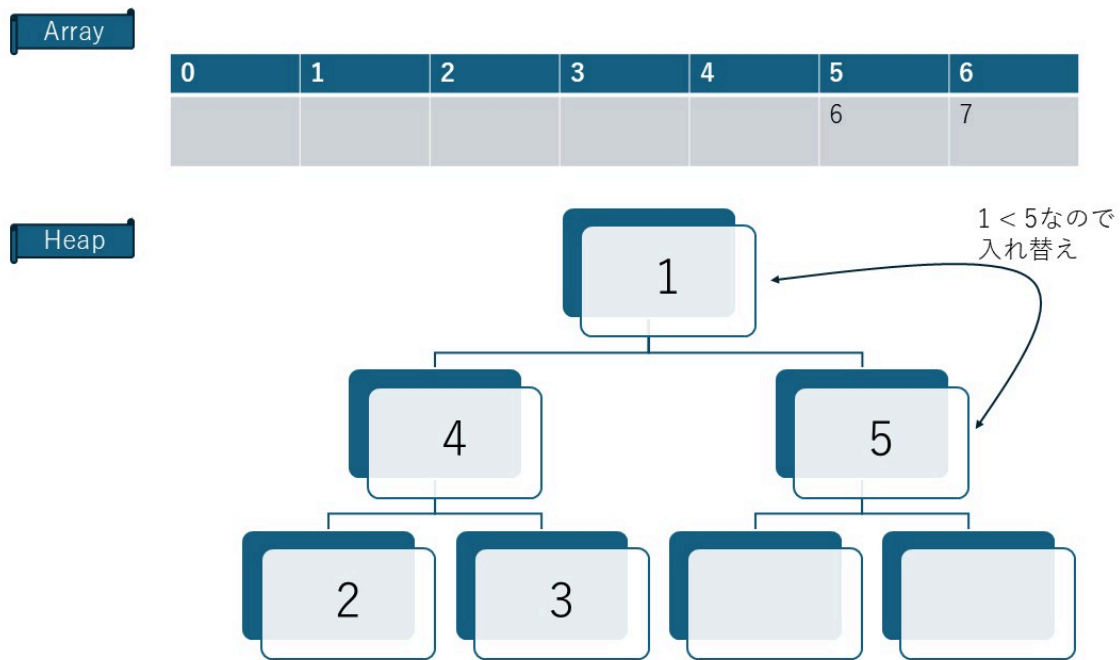
- 頂点の 6 を配列に戻す



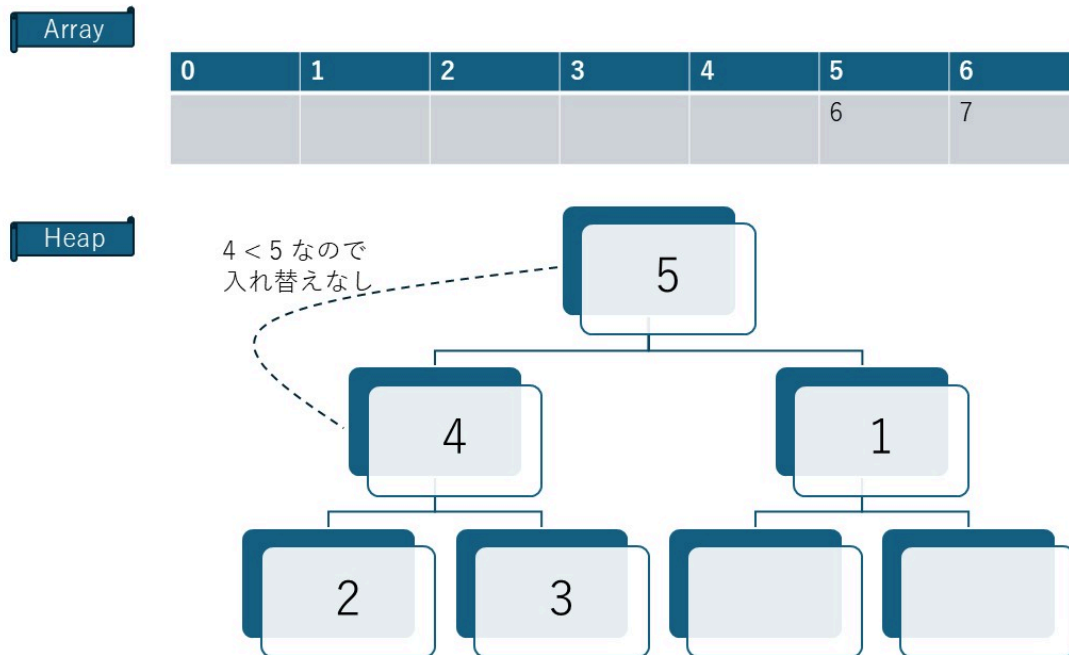
- 右下の 1 を空いた頂点に移動



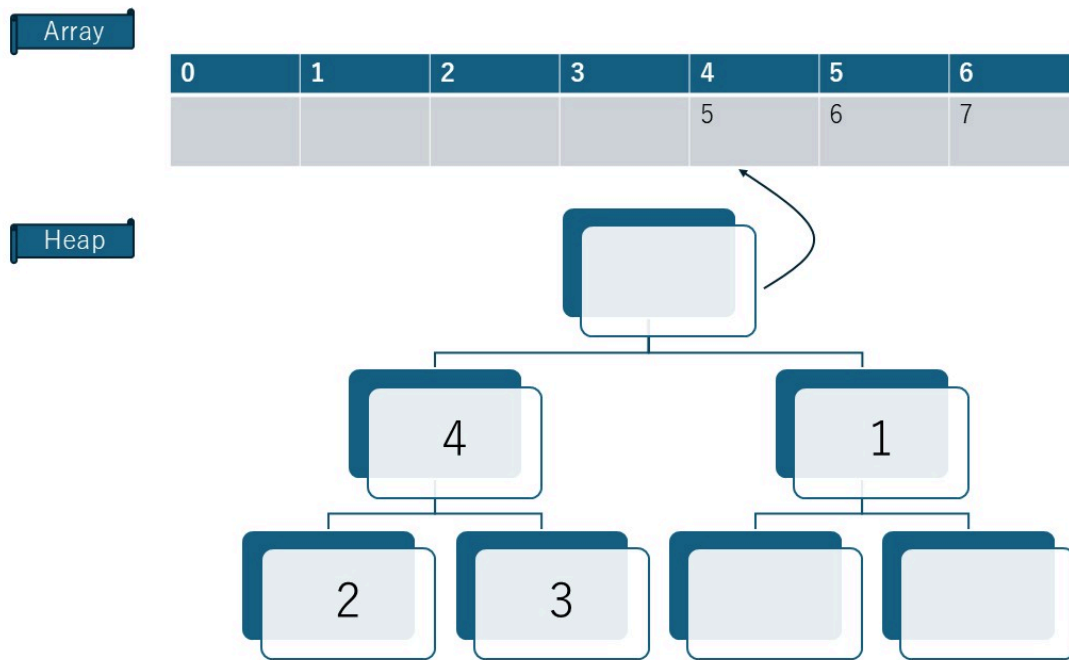
- $1 < 5$ なので、入れ替え



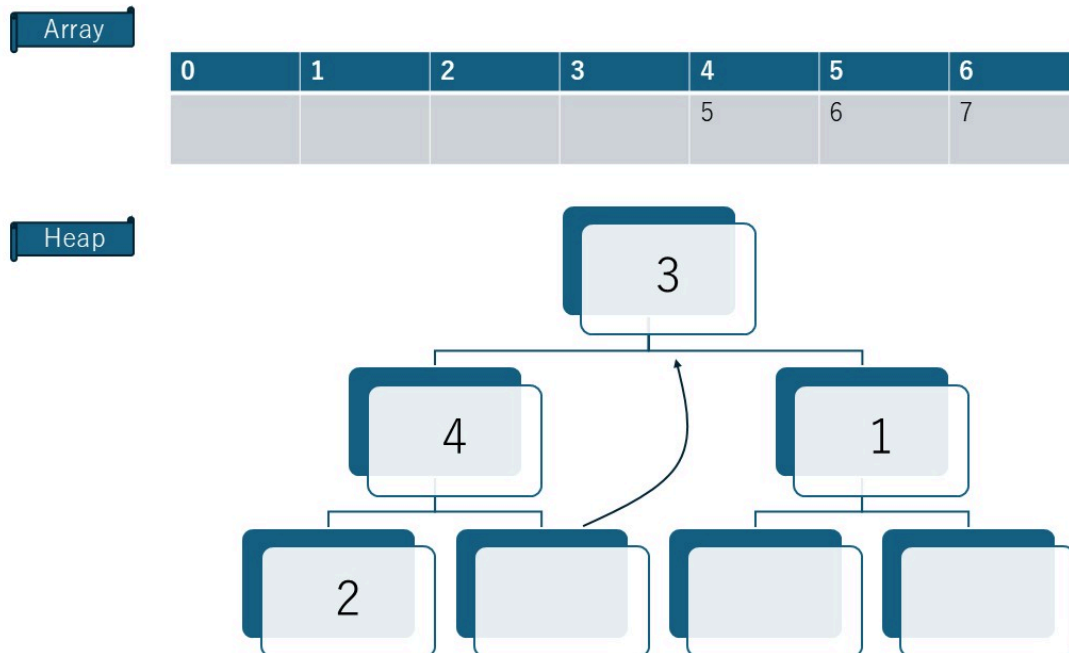
- $4 < 5$ なので、安定



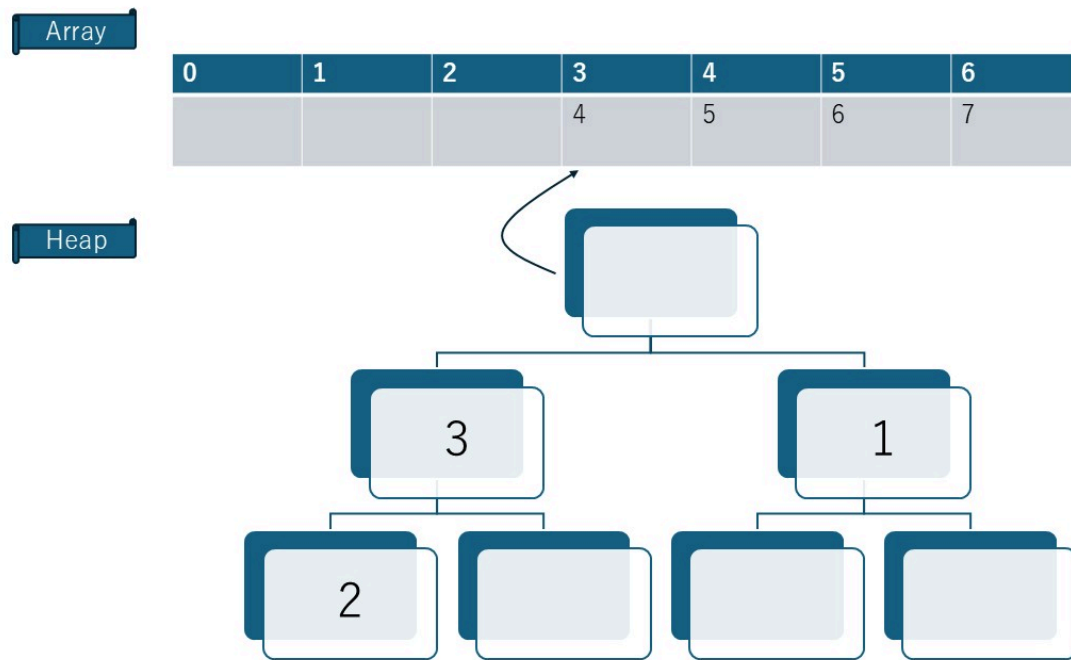
- 頂点の 5 を配列に戻す



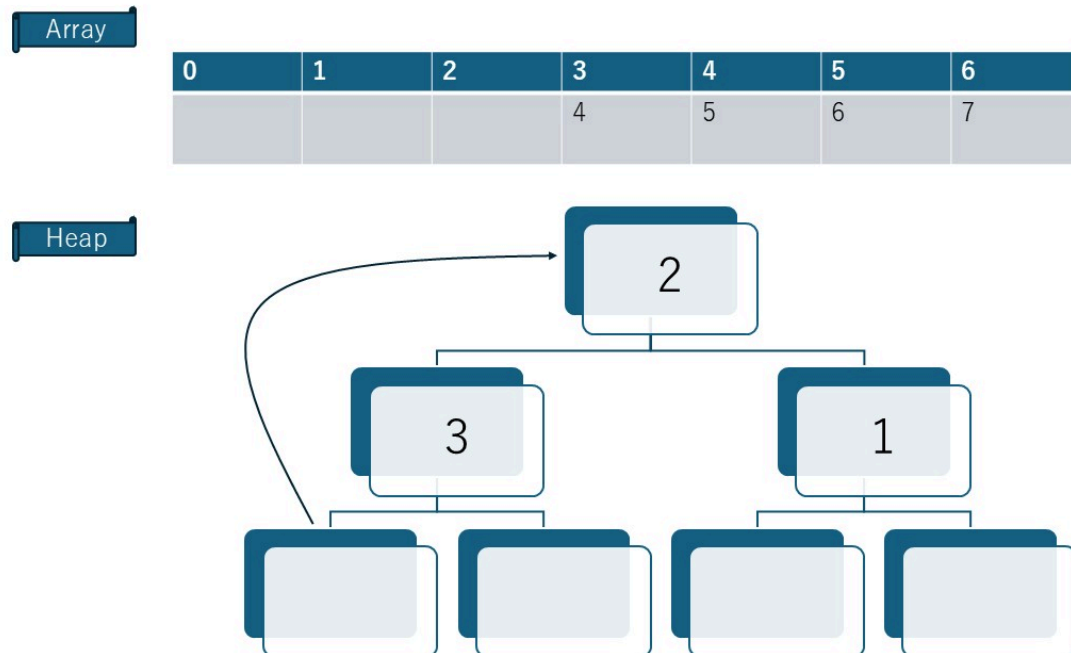
- 右下の 3 を空いた頂点に移動



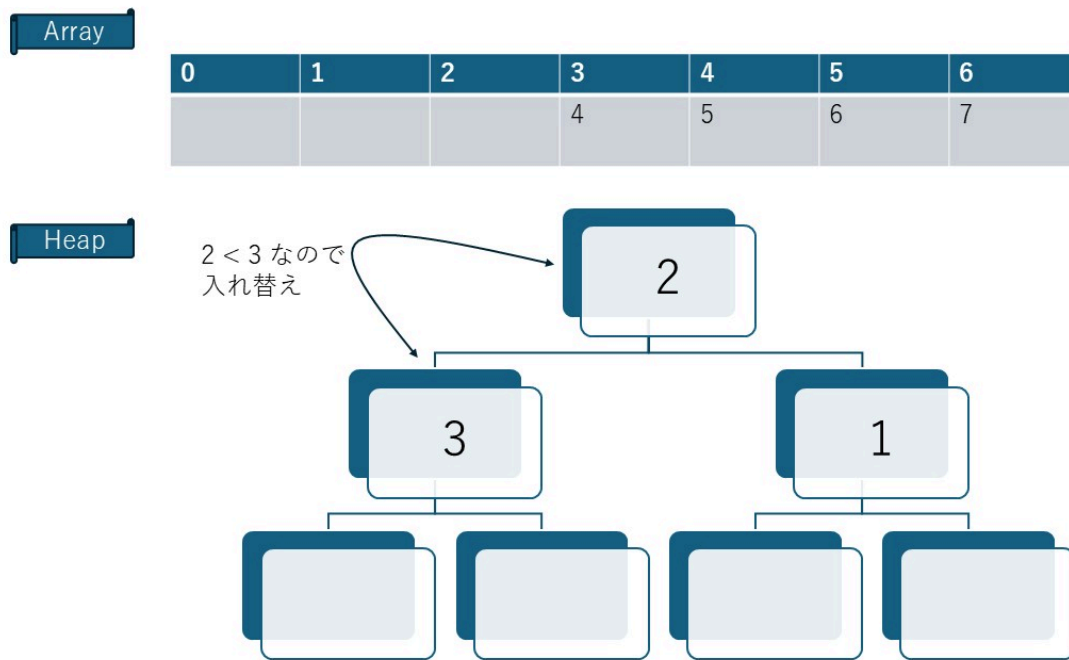
- 頂点の 4 を配列に戻す



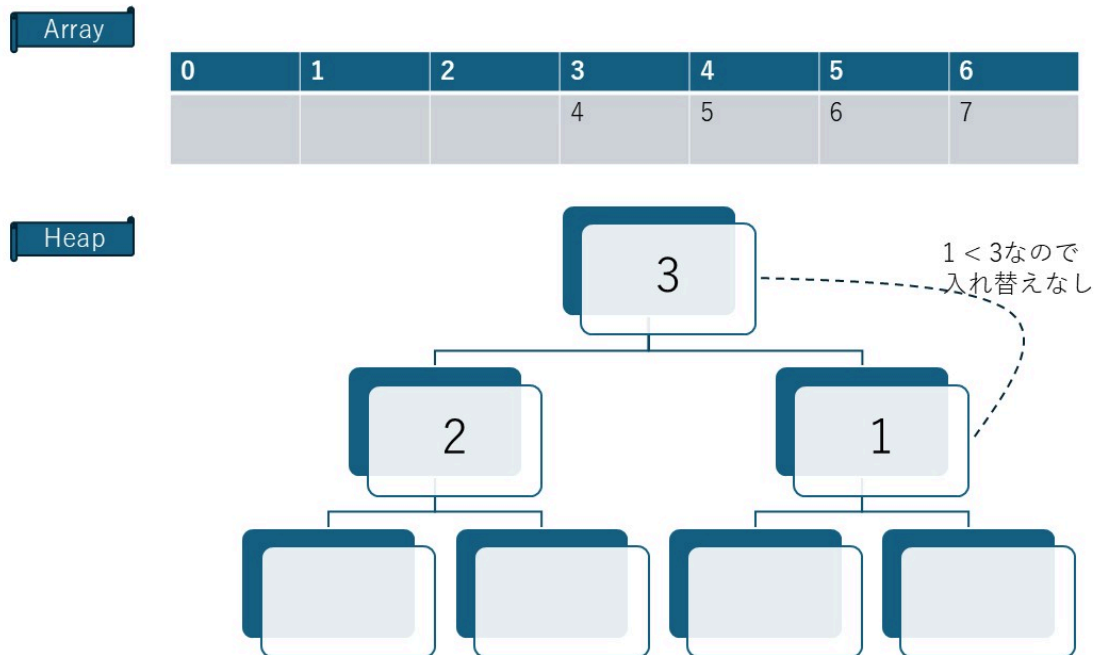
- 右下の 2 を空いた頂点に移動



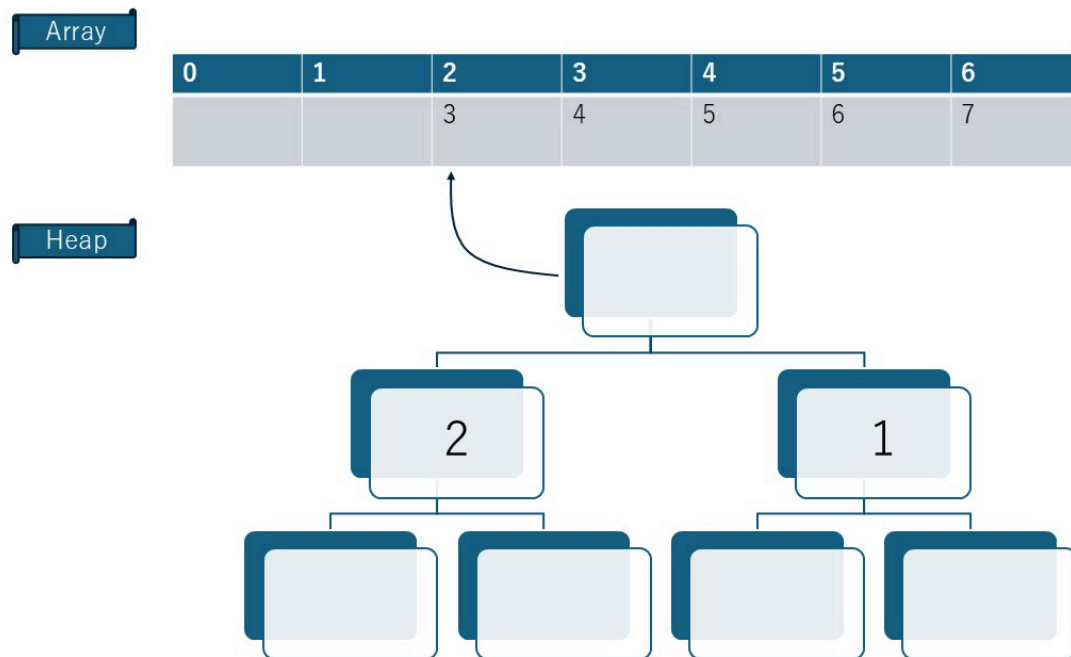
- $2 < 3$ なので、入れ替え



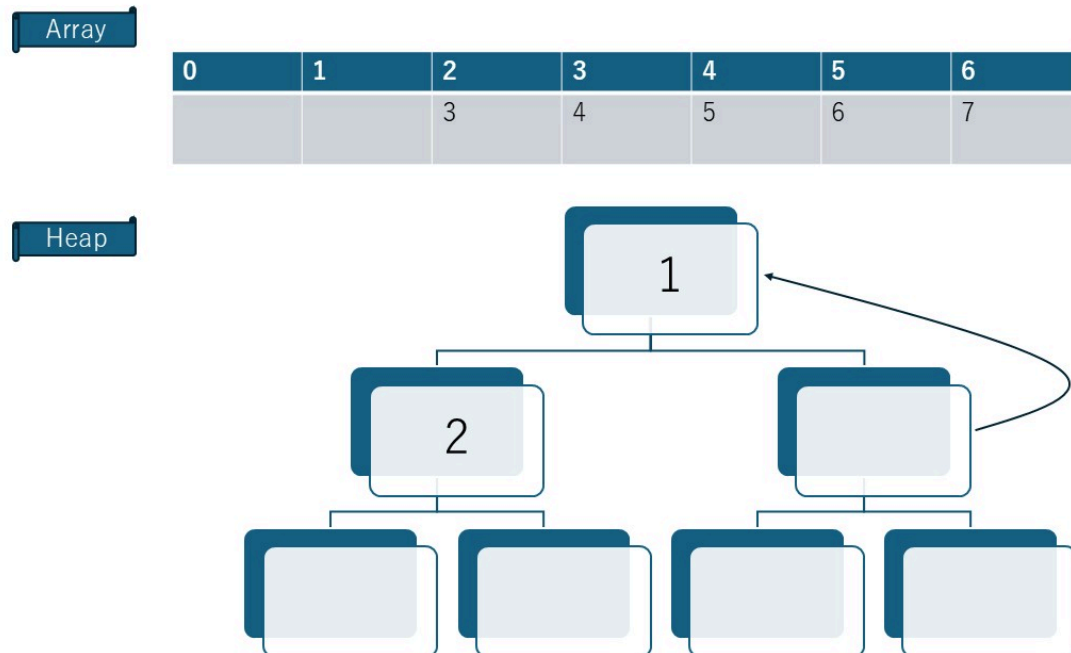
- $1 < 3$ なので、安定



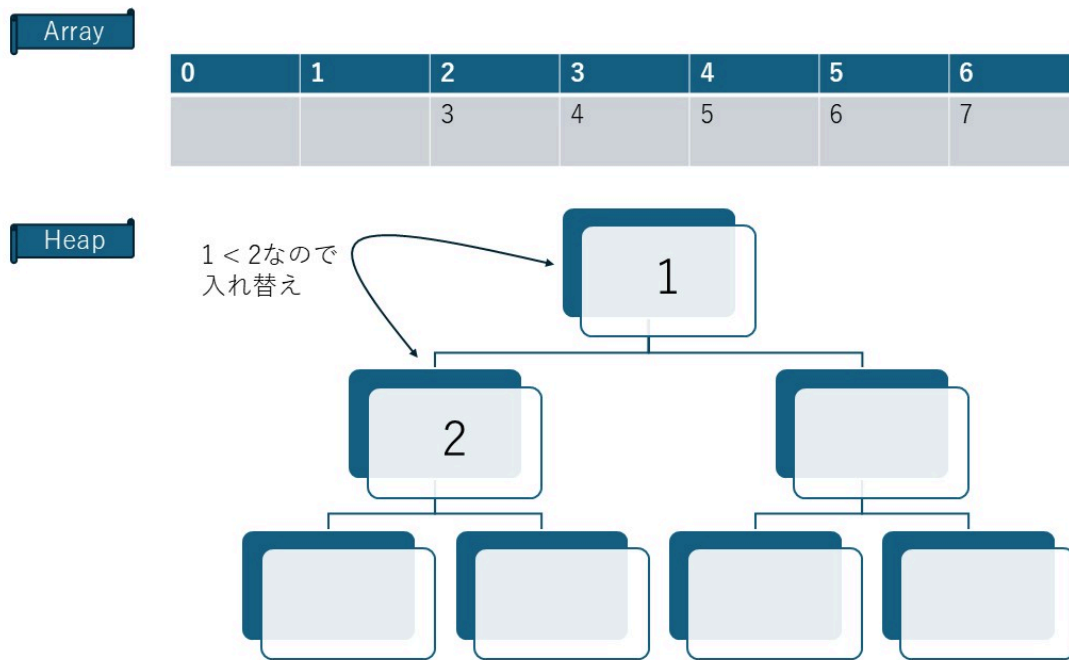
- 頂点の 3 を配列に戻す



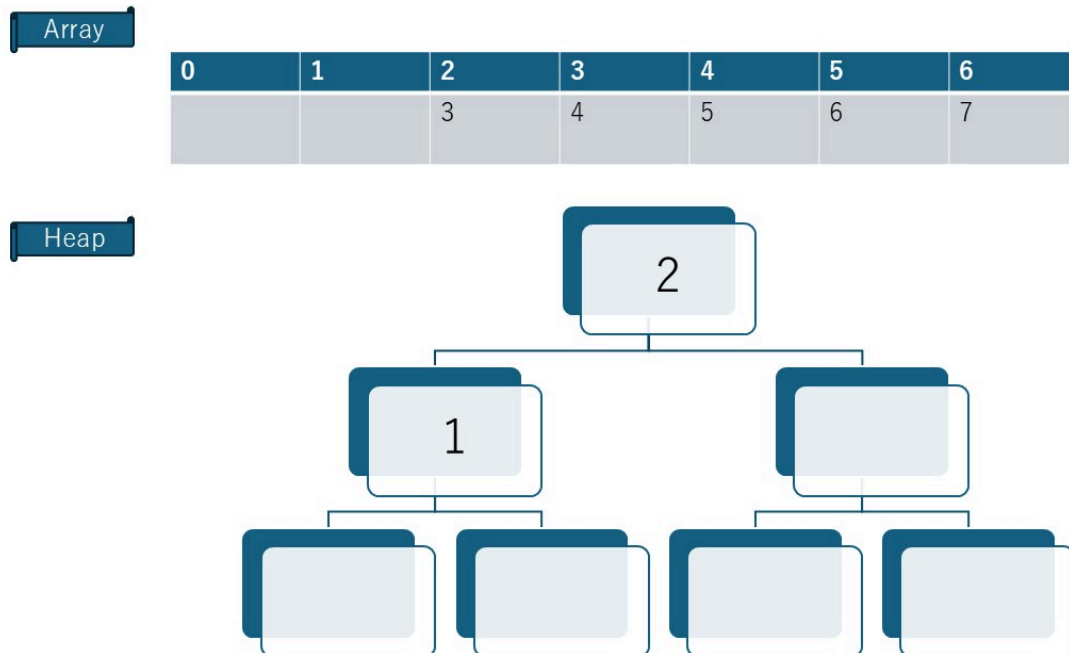
- 右下の 1 を空いた頂点に移動



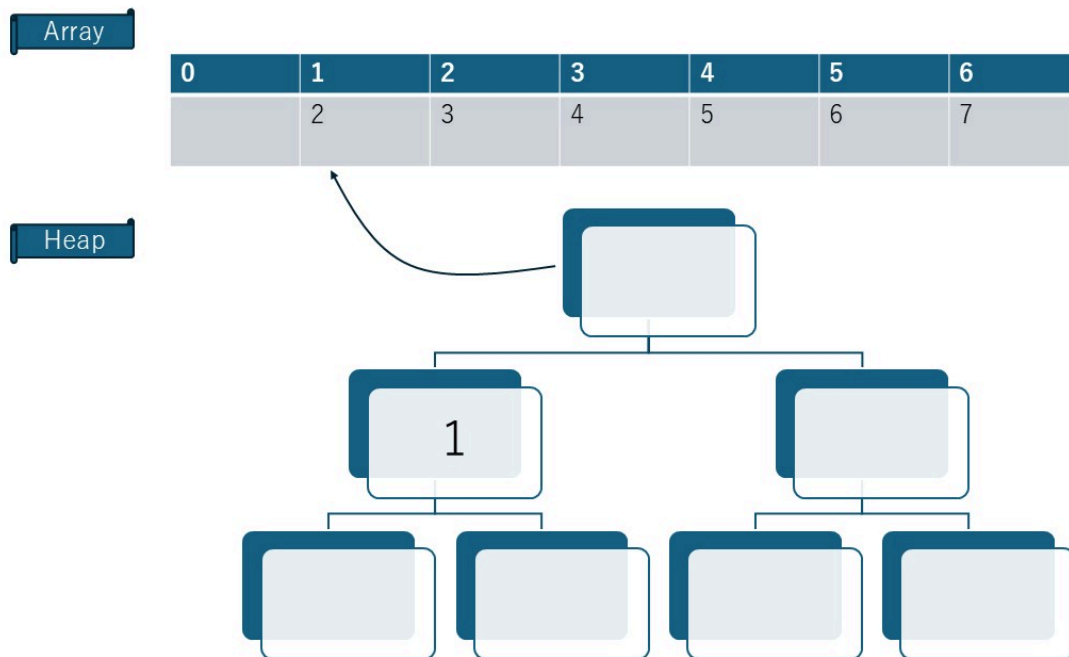
- $1 < 2$ なので、入れ替え



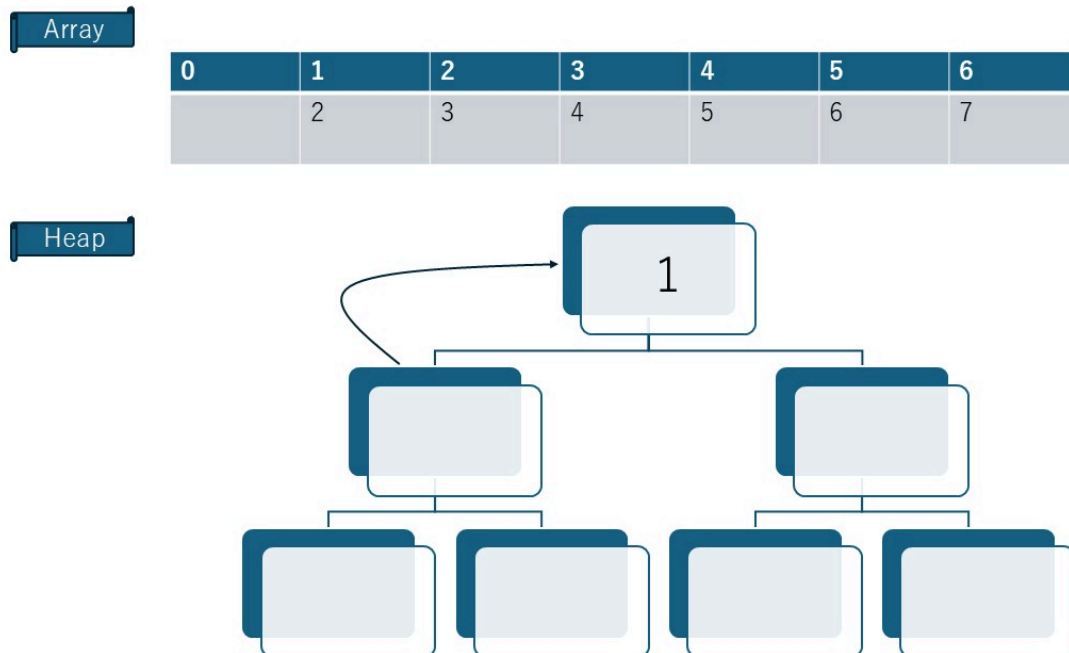
- 安定



- 頂点の 2 を配列にも戻す



- 右下の 1 を空いた頂点に移動



- 頂点の 1 を配列に戻して、完成

